

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

INTERNSHIP 1 REPORT

DEVELOPMENT OF A REPRODUCIBLE CREDIT CARD FRAUD DETECTION PIPELINE

Full name	Dang The Dong A
Student ID	2570791
Major	Computer Science
Course	Internship 1
Instructor	Prof. Quan Thanh Tho
Submission date	30 April 2026

Ho Chi Minh City, 2026

Acknowledgement

I would like to express my sincere appreciation to the lecturers and supervisor of Ho Chi Minh City University of Technology for their guidance during the completion of this Internship 1 project. Their comments and academic direction were valuable in refining both the technical scope and the presentation of the work.

I also acknowledge the open-source community for the software libraries used in this project, particularly Python, scikit-learn, imbalanced-learn, Matplotlib, and Streamlit. These tools provided a reliable foundation for implementing and evaluating the proposed workflow.

Abstract

This report presents the design and implementation of a reproducible machine learning pipeline for credit card fraud detection using the public European cardholder transaction dataset. The objective of the project was not limited to training a classifier; rather, it was to develop a compact engineering workflow that supports data preparation, model training, evaluation, experiment tracking, and result inspection in a systematic manner.

The implemented system includes temporal train-test splitting, feature scaling, class imbalance handling with SMOTE, baseline model training, hyperparameter sweep, run comparison, and a lightweight dashboard for reviewing stored experiment artifacts. Logistic regression and random forest were selected as baseline models because they provide a useful contrast between a linear model and an ensemble model for tabular classification.

Evaluation was carried out using metrics that are appropriate for imbalanced data, namely precision, recall, F1-score, and average precision. Among the saved experimental runs, the random forest model achieved the best overall balance, with precision of 0.8906, recall of 0.7600, F1-score of 0.8201, and average precision of 0.8074. Logistic regression achieved higher recall at 0.8800 but produced substantially lower precision at 0.0676, leading to a significantly weaker F1-score. These results indicate that the proposed pipeline is suitable as an Internship 1 case study because it demonstrates both technical implementation and critical evaluation.

Contents

Acknowledgement	i
Abstract	ii
1 Introduction	1
1.1 Background	1
1.2 Objectives	1
1.3 Scope	2
1.4 Report Organization	2
2 Assigned Tasks and Work Plan	3
2.1 Assigned Tasks	3
2.2 Work Plan	3
2.3 Expected Deliverables	4
3 Theoretical Background	5
3.1 Credit Card Fraud Detection	5
3.2 Imbalanced Classification	5
3.3 Evaluation Metrics	5
3.4 Temporal Holdout Strategy	6
3.5 Baseline Models	6
4 Requirement Analysis and System Design	7
4.1 Functional Requirements	7
4.2 Non-Functional Requirements	7
4.3 Overall Workflow	8
4.4 Module Organization	8

4.5	Artifact Design	8
5	Implementation	9
5.1	Development Environment	9
5.2	Dataset Preparation	9
5.3	Training Workflow	9
5.4	Evaluation Workflow	10
5.5	Hyperparameter Sweep and Comparison Support	10
5.6	Dashboard Interface	10
6	Experimental Setup and Results	11
6.1	Dataset Summary	11
6.2	Model Configurations	11
6.3	Quantitative Results	12
6.4	Precision-Recall Curves	12
6.5	Analysis of Results	13
7	Discussion	14
7.1	Main Contributions of the Project	14
7.2	Limitations	14
7.3	Learning Outcomes	14
7.4	Personal Contribution Summary	15
8	Conclusion and Future Work	16
8.1	Conclusion	16
8.2	Future Work	16
A	Example Commands	17
B	References	18

List of Figures

- 6.1 Precision-recall curve for the logistic regression run 20260429T071627Z-bdf01b47 12
- 6.2 Precision-recall curve for the random forest run 20260429T071850Z-bd9b899b 13

List of Tables

6.1	Prepared dataset summary	11
6.2	Model performance on the temporal holdout set	12

Chapter 1

Introduction

1.1 Background

Credit card fraud detection is a representative problem in applied machine learning, especially in domains where the target class is rare and the cost of misclassification is high. In this setting, a useful solution requires more than a single trained model. It must also incorporate correct preprocessing, leakage prevention, reproducibility, and clear evaluation procedures.

The present project was developed as a compact fraud detection workflow based on the public `creditcard.csv` dataset. The emphasis of the project is placed on sound engineering practice: modular implementation, deterministic execution, explicit artifact management, and evaluation methods that are appropriate for highly imbalanced data.

1.2 Objectives

The objectives of the project are as follows:

1. to develop a modular pipeline for credit card fraud detection;
2. to apply a temporal train-test split in order to reduce information leakage;
3. to perform preprocessing and imbalance handling in a reproducible manner;
4. to compare baseline models using suitable metrics for imbalanced classification;
5. to preserve experiment outputs for subsequent inspection and comparison.

1.3 Scope

The scope of the work includes data preparation, model training, evaluation, hyperparameter sweep, run comparison, and dashboard-based inspection of saved results. The project does not include online inference, large-scale deployment, enterprise integration, or real-time fraud response. Its primary purpose is to demonstrate a complete and well-structured machine learning workflow at the scale appropriate to Internship 1.

1.4 Report Organization

The remainder of this report is organized as follows. Chapter 2 presents the assigned tasks and work plan. Chapter 3 reviews the theoretical background relevant to the project. Chapter 4 discusses the requirements and overall system design. Chapter 5 describes the implementation in detail. Chapter 6 presents the experimental setup and evaluation results. Chapter 7 discusses the significance and limitations of the work. Chapter 8 concludes the report and outlines possible future extensions.

Chapter 2

Assigned Tasks and Work Plan

2.1 Assigned Tasks

The project was carried out through the following main tasks:

1. studying the characteristics of the fraud detection problem and the public dataset used in the project;
2. implementing a preprocessing stage that validates the data, sorts transactions by time, and creates a temporal split;
3. applying feature scaling and oversampling to the training data in a controlled manner;
4. training and evaluating baseline classification models;
5. generating experiment artifacts such as reports, curve data, and image outputs;
6. tracking multiple experimental runs for later comparison;
7. developing a simple dashboard for reviewing saved runs.

2.2 Work Plan

The work was organized into the following stages:

1. problem study and dataset familiarization;
2. preprocessing pipeline design and implementation;
3. model training and evaluation implementation;

4. experiment tracking and comparison support;
5. dashboard development;
6. documentation and report preparation.

2.3 Expected Deliverables

The final output of the project consists of:

- a modular Python implementation under `src/`;
- processed dataset artifacts under `data/`;
- model, report, sweep, and run-history artifacts under `artifacts/`;
- a Streamlit dashboard under `ui/app.py`;
- project documentation in `README.md`.

Chapter 3

Theoretical Background

3.1 Credit Card Fraud Detection

Credit card fraud detection aims to distinguish fraudulent transactions from legitimate ones in a highly skewed dataset. In practice, the number of fraudulent transactions is very small compared with the number of legitimate transactions. This imbalance makes the problem challenging because a model may achieve a high overall accuracy while still failing to detect many fraud cases.

3.2 Imbalanced Classification

In imbalanced classification, the minority class is underrepresented relative to the majority class. Standard learning algorithms often become biased toward the majority class, especially when the class distribution is extreme. In the context of fraud detection, such bias is undesirable because missed fraud cases may be costly.

To mitigate this issue, the project applies SMOTE to the training partition. SMOTE creates synthetic samples for the minority class in feature space, thereby improving the representation of fraudulent transactions during model fitting.

3.3 Evaluation Metrics

Accuracy alone is not sufficient for this problem. For that reason, the project uses the following evaluation measures:

- **Precision:** the proportion of predicted fraud cases that are truly fraudulent;
- **Recall:** the proportion of actual fraud cases that are correctly detected;

- **F1-score**: the harmonic mean of precision and recall;
- **Average precision**: a summary of model quality across decision thresholds based on the precision-recall curve.

These metrics provide a more meaningful assessment in imbalanced settings, where false negatives and false positives do not have the same practical impact.

3.4 Temporal Holdout Strategy

The project uses the `Time` column to construct a temporal split. Earlier observations are used for training, while later observations are reserved for testing. This choice is more realistic than a random split because it approximates the way a deployed model would be evaluated on future transactions.

3.5 Baseline Models

Two baseline models are considered in this project:

1. **Logistic Regression**, which serves as a simple and interpretable linear baseline for tabular data;
2. **Random Forest**, which is an ensemble method capable of capturing nonlinear relationships and feature interactions.

Chapter 4

Requirement Analysis and System Design

4.1 Functional Requirements

The system is required to perform the following functions:

- load a local dataset or download the default dataset automatically;
- validate the expected columns and sort transactions by time;
- split the dataset into training and test partitions according to temporal order;
- scale input features using statistics learned only from the training partition;
- resample the training data using SMOTE;
- train one or more classification models;
- evaluate saved models on an untouched temporal holdout set;
- conduct hyperparameter sweeps on an inner validation split;
- compare stored runs and display their results through a dashboard.

4.2 Non-Functional Requirements

The most important non-functional requirements are reproducibility, modularity, maintainability, and usability. These are addressed through a separated `src/` structure, a fixed random seed, explicit artifact persistence, run manifests, and a command-line interface that supports repeated execution.

4.3 Overall Workflow

The system is organized into six user-facing stages:

1. `prepare`;
2. `train`;
3. `evaluate`;
4. `sweep`;
5. `compare`;
6. `dashboard`.

The first four stages create experimental outputs, while the last two stages support inspection and comparison of previously saved runs.

4.4 Module Organization

The implementation is divided into specialized modules:

- `src/data_processing.py` manages dataset loading, validation, temporal splitting, scaling, resampling, and metadata persistence;
- `src/model.py` defines the supported model families and sweep search spaces;
- `src/pipeline.py` coordinates the main workflows for training, evaluation, sweeping, and comparison;
- `src/evaluation.py` computes performance metrics and saves precision-recall artifacts;
- `src/cli.py` exposes the workflow through command-line commands;
- `ui/app.py` implements a dashboard for reviewing stored runs.

4.5 Artifact Design

Outputs are organized into separate directories for processed data, trained models, reports, sweeps, and run manifests. This structure prevents accidental overwriting of previous experiments and supports systematic comparison across runs.

Chapter 5

Implementation

5.1 Development Environment

The project is implemented in Python and relies on standard libraries for data processing and machine learning, including Pandas, scikit-learn, imbalanced-learn, Matplotlib, Joblib, and Streamlit. Dependency installation is specified in `requirements.txt`.

5.2 Dataset Preparation

The input dataset contains anonymized numerical features together with the `Time`, `Amount`, and `Class` columns. During preparation, the system validates the required schema, sorts transactions by `Time`, creates the temporal split, applies feature scaling, and stores the processed data and metadata on disk.

The prepared dataset used in this report contains 227,845 training rows and 56,962 test rows. After resampling, the training partition contains 284,285 rows. The target variable is `Class`, and the feature set contains 30 columns, including `Time`, `Amount`, and the anonymized variables `V1` to `V28`.

5.3 Training Workflow

The training stage loads the prepared training data, constructs the requested model with the specified hyperparameters, fits the model, and stores the resulting artifact under a unique run directory. A training summary file is also generated to record the resolved parameters, training size, feature count, and random seed. This mechanism is important for repeatability and later analysis.

5.4 Evaluation Workflow

The evaluation stage loads a stored model, generates fraud scores for the temporal holdout set, applies a decision threshold, and computes precision, recall, F1-score, and average precision. In addition, it saves a JSON evaluation report, a precision-recall curve in CSV form, a PNG figure, an updated run manifest, and an updated run-history index.

5.5 Hyperparameter Sweep and Comparison Support

The sweep stage creates an inner temporal validation split from the training timeline and evaluates candidate hyperparameter combinations according to average precision, F1-score, and recall. The comparison stage then aggregates all completed runs into a sortable summary table. This design makes it possible to inspect the evolution of experiments without reopening each run manually.

5.6 Dashboard Interface

The project includes a lightweight Streamlit dashboard that reads the saved run history and manifests. The dashboard displays overall run statistics, detailed information for individual runs, and side-by-side comparisons. Although it is not essential for the core pipeline, it improves the usability of the project and supports demonstration in an academic setting.

Chapter 6

Experimental Setup and Results

6.1 Dataset Summary

Table 6.1 summarizes the main dataset settings used in the prepared data.

Table 6.1: Prepared dataset summary

Item	Value
Training rows	227,845
Test rows	56,962
Resampled training rows	284,285
Feature count	30
Target column	<code>Class</code>
Temporal split start of test set	145248.0
Sampling strategy	0.25
Random seed	42
Fraud cases in evaluated test set	75

6.2 Model Configurations

The experiments discussed in this report include the following representative evaluated runs:

- **Logistic regression:** `C = 0.5`, `max_iter = 3000`, `solver = liblinear`, and `class_weight = balanced`;
- **Random forest:** `n_estimators = 500`, `max_depth = 12`, `min_samples_split = 4`, and `class_weight = balanced_subsample`.

Both models were evaluated at the threshold value of 0.5 on the same temporal holdout partition.

6.3 Quantitative Results

Table 6.2 presents the principal evaluation metrics extracted from the saved comparison report.

Table 6.2: Model performance on the temporal holdout set

Model	Precision	Recall	F1-score	Average Precision
Logistic Regression	0.0676	0.8800	0.1255	0.7706
Random Forest	0.8906	0.7600	0.8201	0.8074

6.4 Precision-Recall Curves

Figures 6.1 and 6.2 illustrate the saved precision-recall curves for the representative logistic regression and random forest runs.

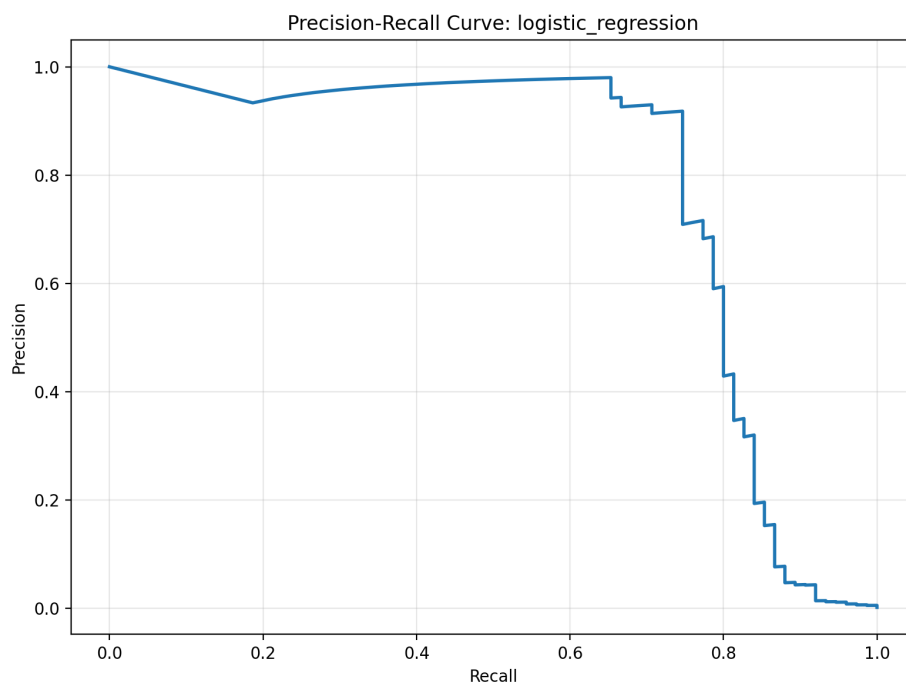


Figure 6.1: Precision-recall curve for the logistic regression run 20260429T071627Z-bdf01b47

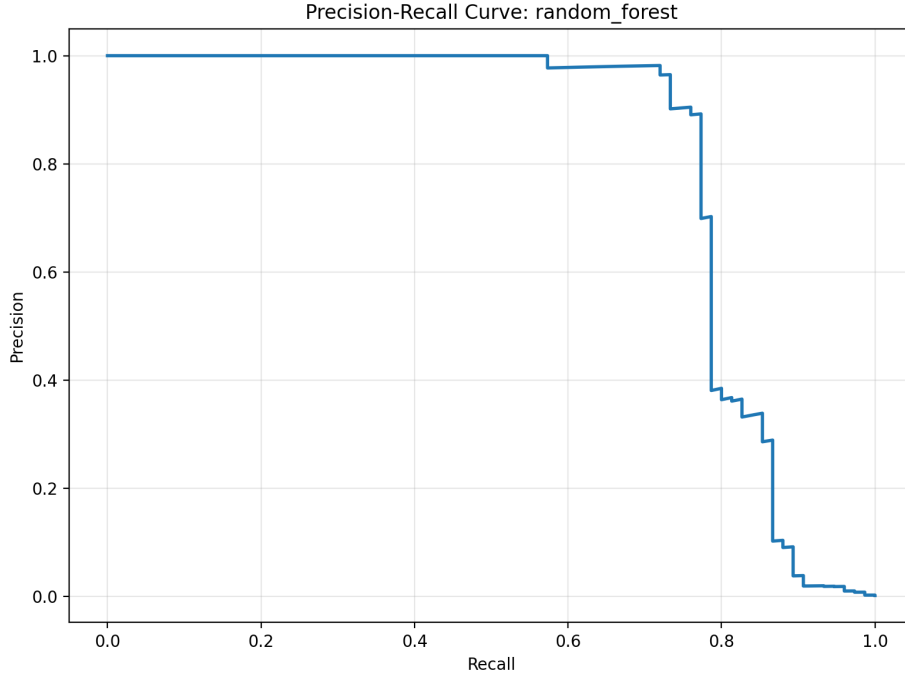


Figure 6.2: Precision-recall curve for the random forest run 20260429T071850Z-bd9b899b

6.5 Analysis of Results

The logistic regression model achieved a recall of 0.8800, indicating that most fraud cases in the test set were detected. However, its precision was only 0.0676 because the model predicted 977 positive cases while the test set contained only 75 actual fraud cases. This large number of false positives substantially reduced the practical usefulness of the model at the chosen threshold.

The random forest model produced a more balanced outcome. It achieved precision of 0.8906 and recall of 0.7600, resulting in an F1-score of 0.8201 and an average precision of 0.8074. Among the evaluated runs recorded in the comparison report, this configuration provided the most favorable tradeoff between detecting fraud and limiting false alarms.

Chapter 7

Discussion

7.1 Main Contributions of the Project

The main contribution of this project is the development of a pipeline-based system that makes the fraud detection workflow easier to execute and manage. Instead of training and evaluating models through scattered scripts or notebooks, the project organizes the process into a clear sequence of stages, including data preparation, model training, evaluation, hyperparameter sweep, and result comparison. This design improves usability, reproducibility, and experiment management, allowing users to run and review machine learning experiments more efficiently.

7.2 Limitations

Despite its strengths, the project has several limitations. Only two baseline model families were studied. The current workflow does not include advanced feature engineering, systematic threshold optimization, or model explainability. In addition, the solution remains an offline batch workflow and is not designed as a production fraud monitoring service.

7.3 Learning Outcomes

Through the completion of this project, several technical skills were strengthened, including preprocessing for tabular data, evaluation of imbalanced classifiers, experiment management, artifact organization, and the presentation of machine learning results in a structured academic format.

7.4 Personal Contribution Summary

The primary contributions made in this project include the design of the preprocessing workflow, the implementation of training and evaluation stages, the introduction of run tracking and comparison artifacts, and the development of a dashboard for reviewing saved experiments. These contributions extend beyond simple model usage and reflect basic machine learning system design and engineering practice.

Chapter 8

Conclusion and Future Work

8.1 Conclusion

This report has presented the development of a reproducible credit card fraud detection pipeline for Internship 1. The implemented workflow covers data preparation, temporal splitting, feature scaling, oversampling, model training, evaluation, hyperparameter sweep, run comparison, and dashboard-based inspection. Experimental results indicate that the random forest configuration provides the best balance among the evaluated runs, whereas logistic regression offers higher recall at the cost of a large number of false positives.

Taken as a whole, the project demonstrates an appropriate level of technical depth for Internship 1 by combining problem formulation, implementation, reproducibility, experiment management, and performance analysis within a single coherent workflow.

8.2 Future Work

Several directions may be considered for future improvement:

- evaluating stronger model families such as gradient boosting methods;
- performing systematic threshold tuning based on application-specific cost tradeoffs;
- incorporating explainability techniques to interpret model decisions;
- extending the system into a service-oriented architecture or API;
- adding broader automated tests for pipeline reliability.

Appendix A

Example Commands

The following commands illustrate a typical execution flow:

```
python -m src.cli prepare
python -m src.cli train --model logistic_regression --lr-c 0.5 --lr-max-iter 3000
python -m src.cli evaluate --model logistic_regression
python -m src.cli train --model random_forest --rf-n-estimators 500 --rf-max-depth 12
python -m src.cli evaluate --model random_forest
python -m src.cli compare --output-path artifacts/comparison_report.json
streamlit run ui/app.py
```

Appendix B

References

Bibliography

- [1] TensorFlow. Credit card fraud detection dataset mirror. <https://storage.googleapis.com/download.tensorflow.org/data/creditcard.csv>
- [2] scikit-learn developers. scikit-learn documentation. <https://scikit-learn.org/stable/>
- [3] imbalanced-learn developers. imbalanced-learn documentation. <https://imbalanced-learn.org/stable/>
- [4] Streamlit. Streamlit documentation. <https://docs.streamlit.io/>